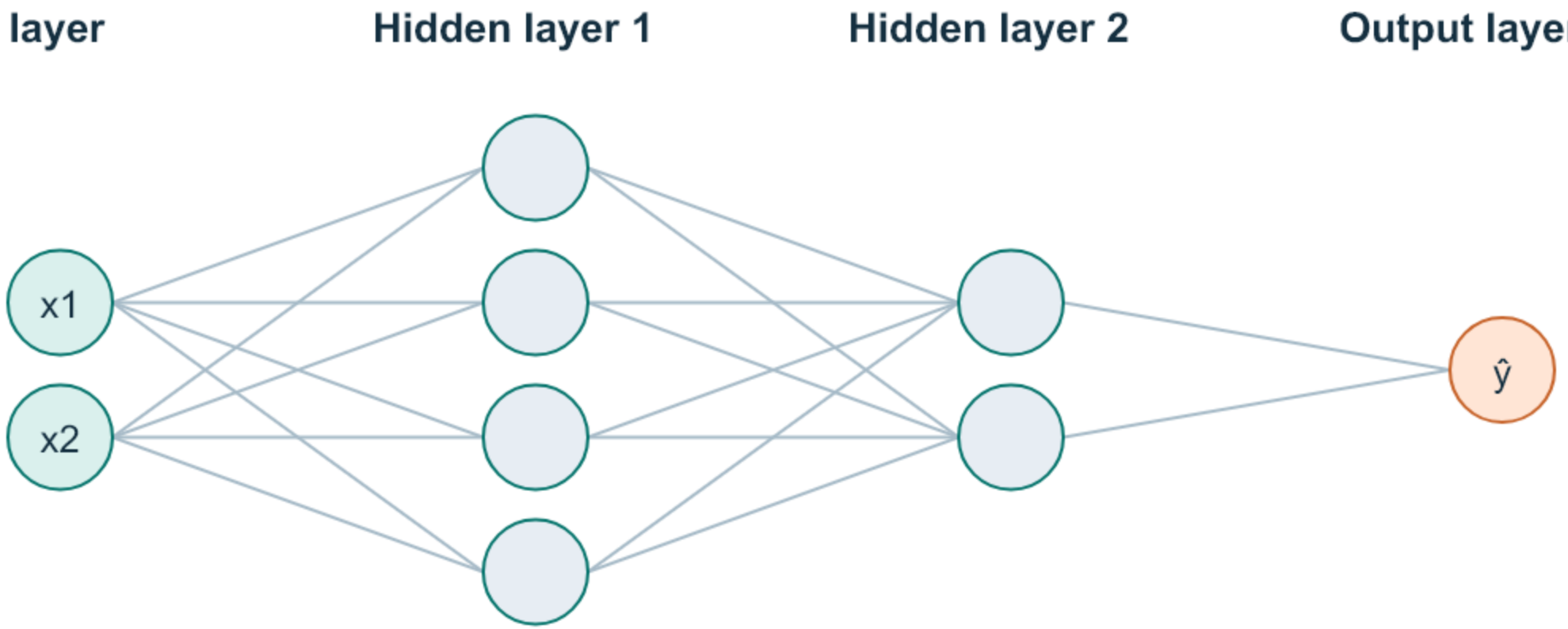
Neural network components and learning

Yadunandan Mandalaneni
Model Development
AIML 501 Assignment 2.4

Structure, computation, and evidence from TensorFlow Playground

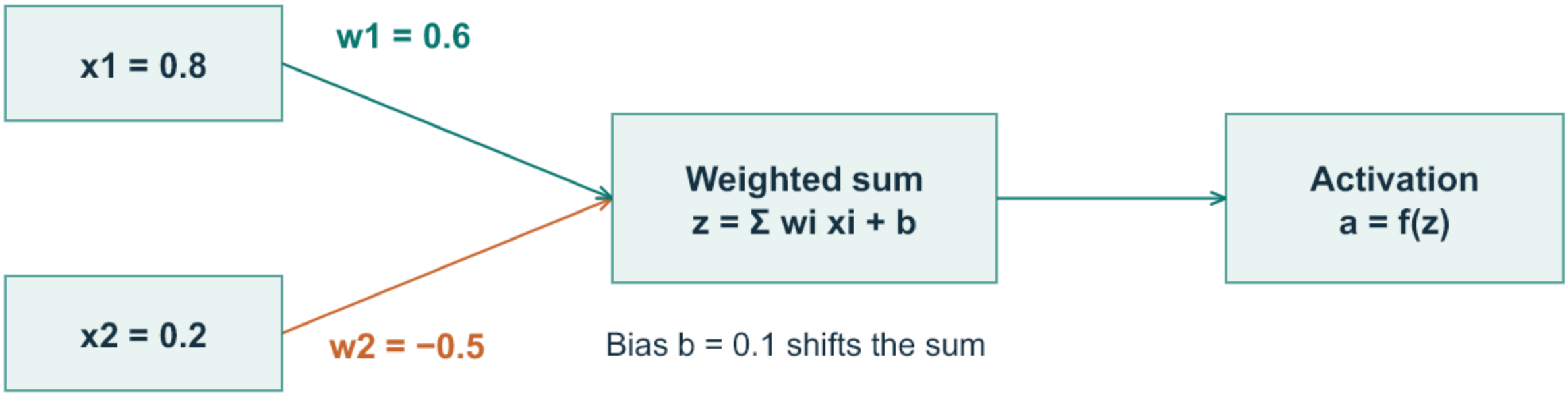
Architecture and data flow



Features enter at the left. Each layer passes its activations to the next layer.

Architecture shown 2 inputs, hidden widths 4 and 2, one output. Biases omitted from the drawing.

Neurons weights and bias



Illustrative calculation

$z = (0.6 \times 0.8) + (-0.5 \times 0.2) + 0.1 = 0.48$

$a = \tanh(0.48) \approx 0.446$

Layers and model capacity

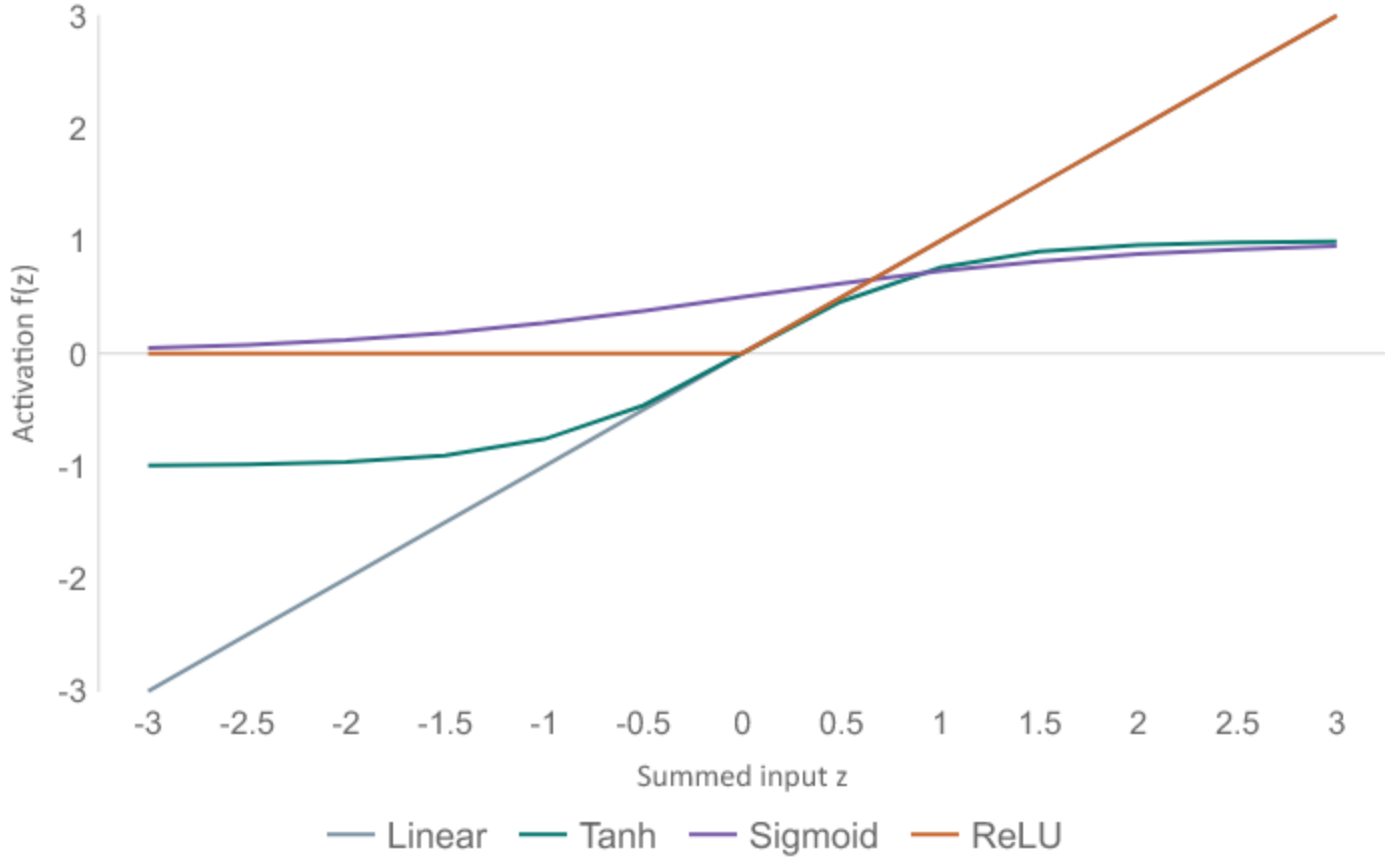
Depth counts hidden stages. Width counts the neurons in each stage.

Observed architecture	Trainable parameters	Runs
2 inputs, 4 then 2 hidden, 1 output	25	A B C
2 raw inputs, 2 hidden, 1 output	9	E F
2 squared inputs, no hidden, 1 output	3	D

Count for the larger network

$$(2 + 1) \times 4 + (4 + 1) \times 2 + (2 + 1) \times 1 = 25$$

Activation functions



Linear f(z) = z
Tanh range −1 to 1
Sigmoid range 0 to 1
ReLU max(0, z)

The menu changes hidden activations. Playground classification keeps tanh at the output.

Loss functions

Loss measures prediction error

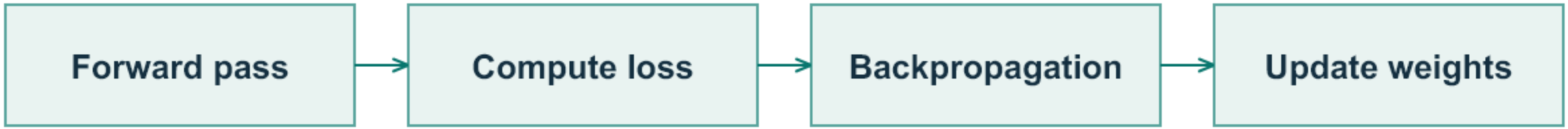
Playground
L = average of ½(ŷ − y)²
Class targets are −1 and +1

Illustrative example
Target y = +1 and prediction ŷ = 0.6
Loss = ½(0.6 − 1)² = 0.08

Binary cross entropy
For probability p and label y in {0, 1}
−[y log(p) + (1 − y) log(1 − p)]

Training loss guides fitting. Held-out test loss checks performance on other samples.

Optimization algorithms



Gradient step θnew = θold − learning rate × gradient

Algorithm	Update behavior
Mini-batch gradient descent	Uses an averaged gradient from a batch
Momentum SGD	Carries a velocity based on earlier gradients
Adam	Adapts updates using first and second gradient moments

Observed runs use learning rate 0.03 and batch size 10. Adam is a conceptual comparison.

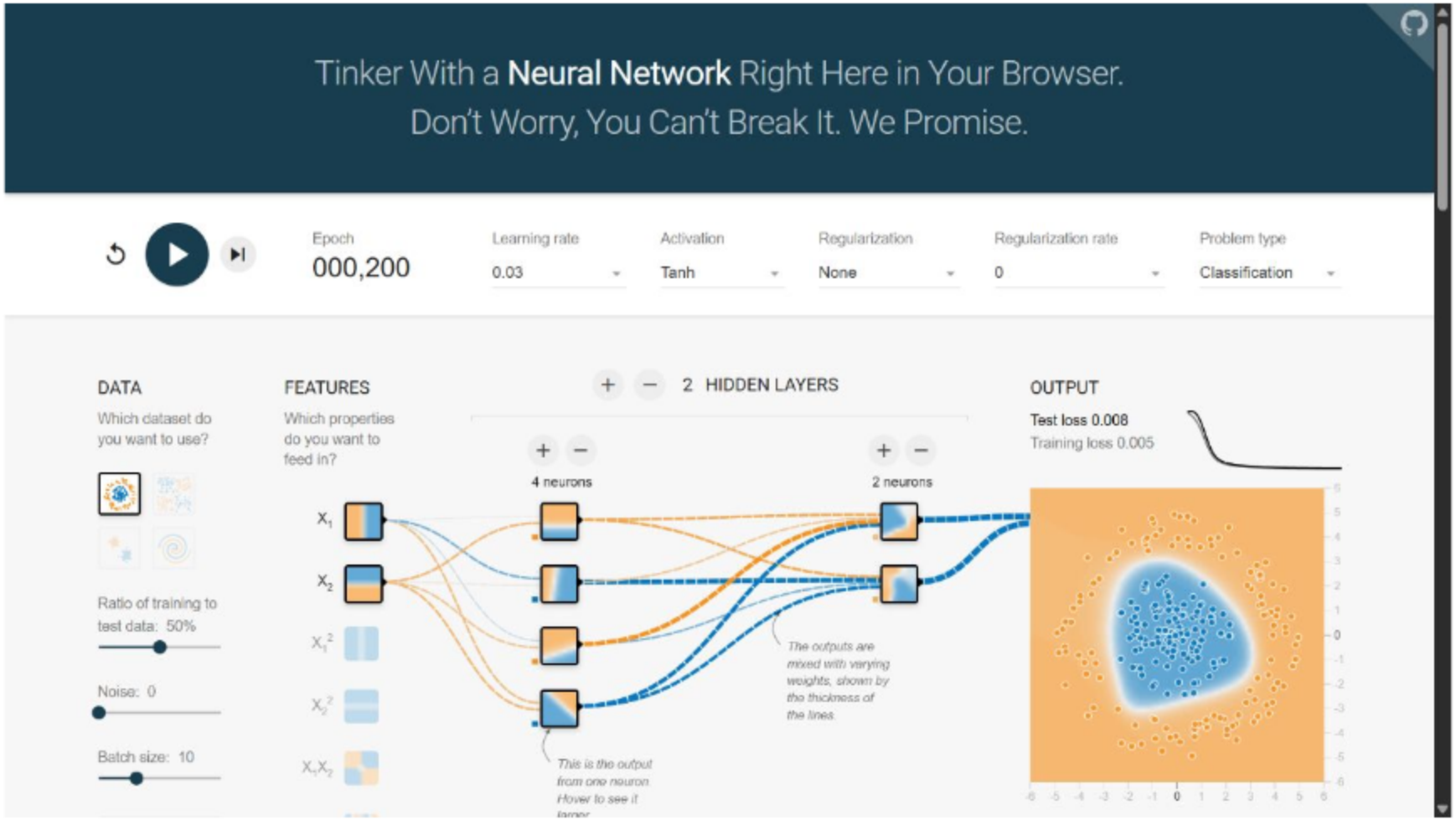
Playground experiment design

Run	Data	Noise	Hidden widths	Inputs	Activation	Train / test loss
A	Circle	0	4, 2	x1 x2	linear	0.485 / 0.541
B	Circle	0	4, 2	x1 x2	tanh	0.005 / 0.008
C	Circle	30	4, 2	x1 x2	tanh	0.118 / 0.159
D	Circle	0	None	$x1^2$ $x2^2$	N/A	0.003 / 0.002
E	Circle	0	2	x1 x2	tanh	0.218 / 0.267
F	Gaussian	0	2	x1 x2	tanh	0.000 / 0.000

Shared budget 200 epochs
Learning rate 0.03, batch size 10, 50% training data, no regularization

Single runs with recorded seeds. Loss values are rounded. Data changes can also change the sample.

Nonlinearity changes the circle fit

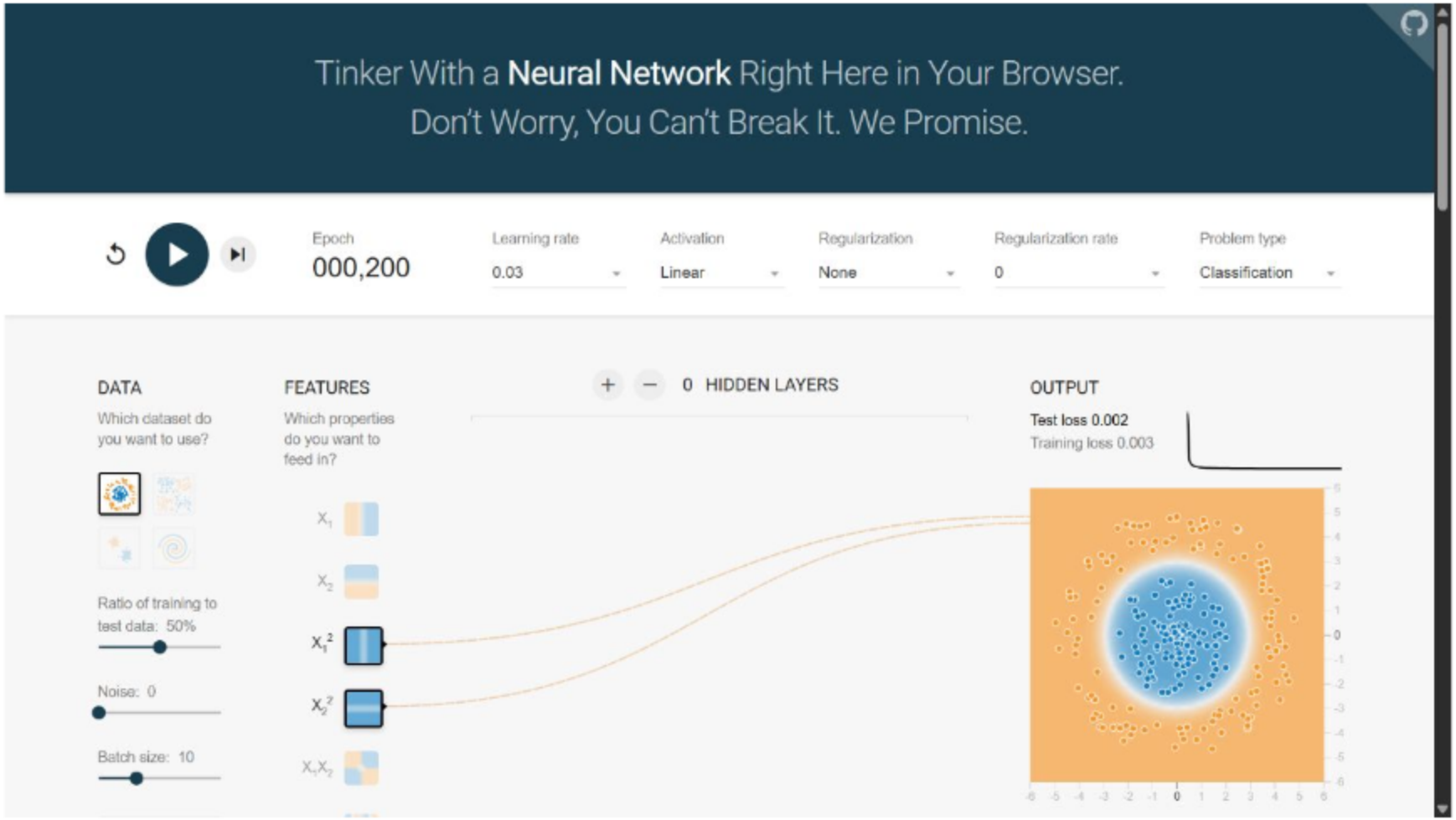


Linear hidden units
Train 0.485
Test 0.541

Tanh hidden units
Train 0.005
Test 0.008

Run B at 200 epochs. The curve encloses the central class.

Features can simplify the architecture



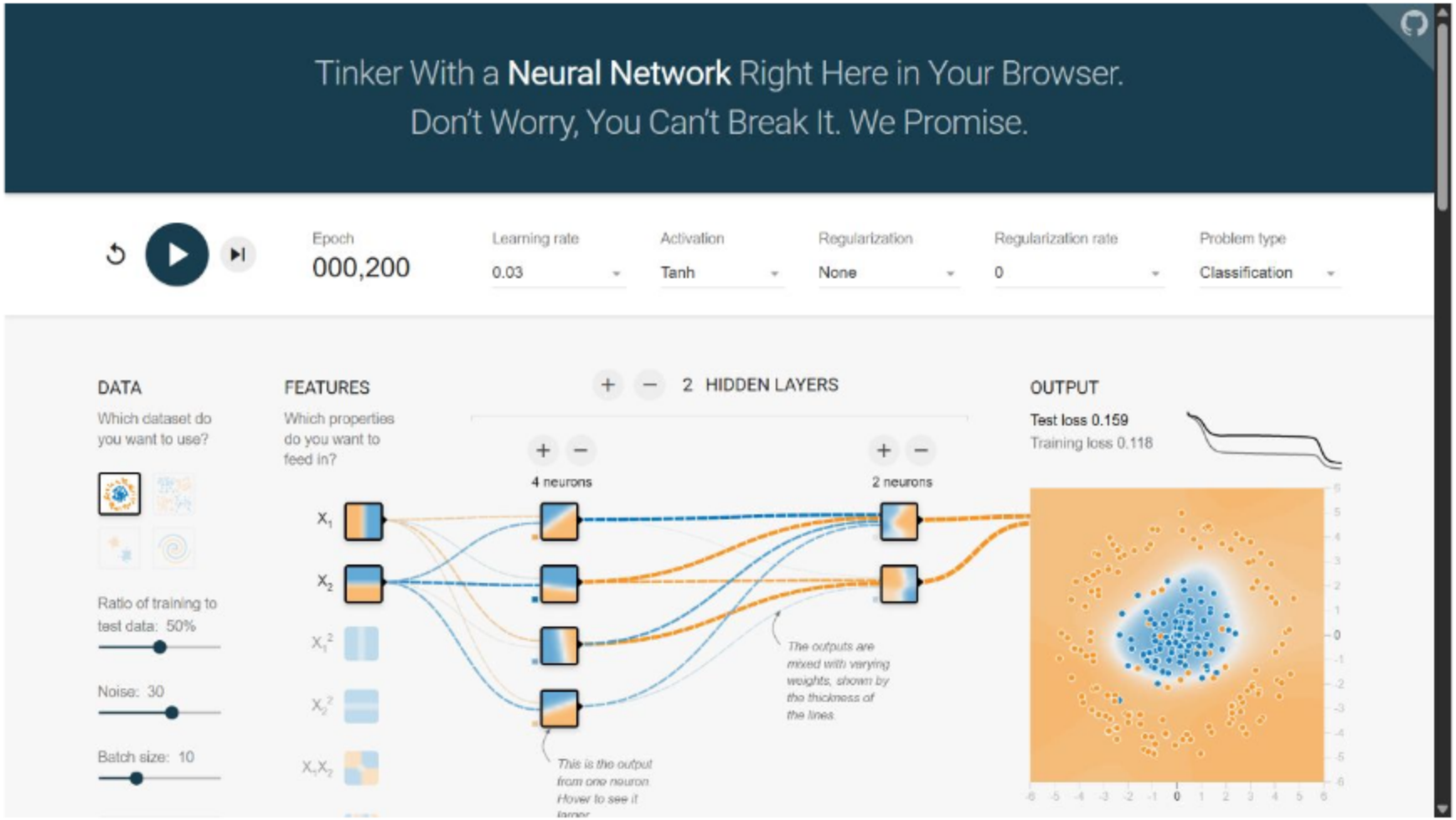
Squared inputs
$x1^2$ and $x2^2$

No hidden layer
3 parameters

Train 0.003
Test 0.002

A sum of squared coordinates can describe an enclosed boundary in the original input space.

Noise and limited capacity

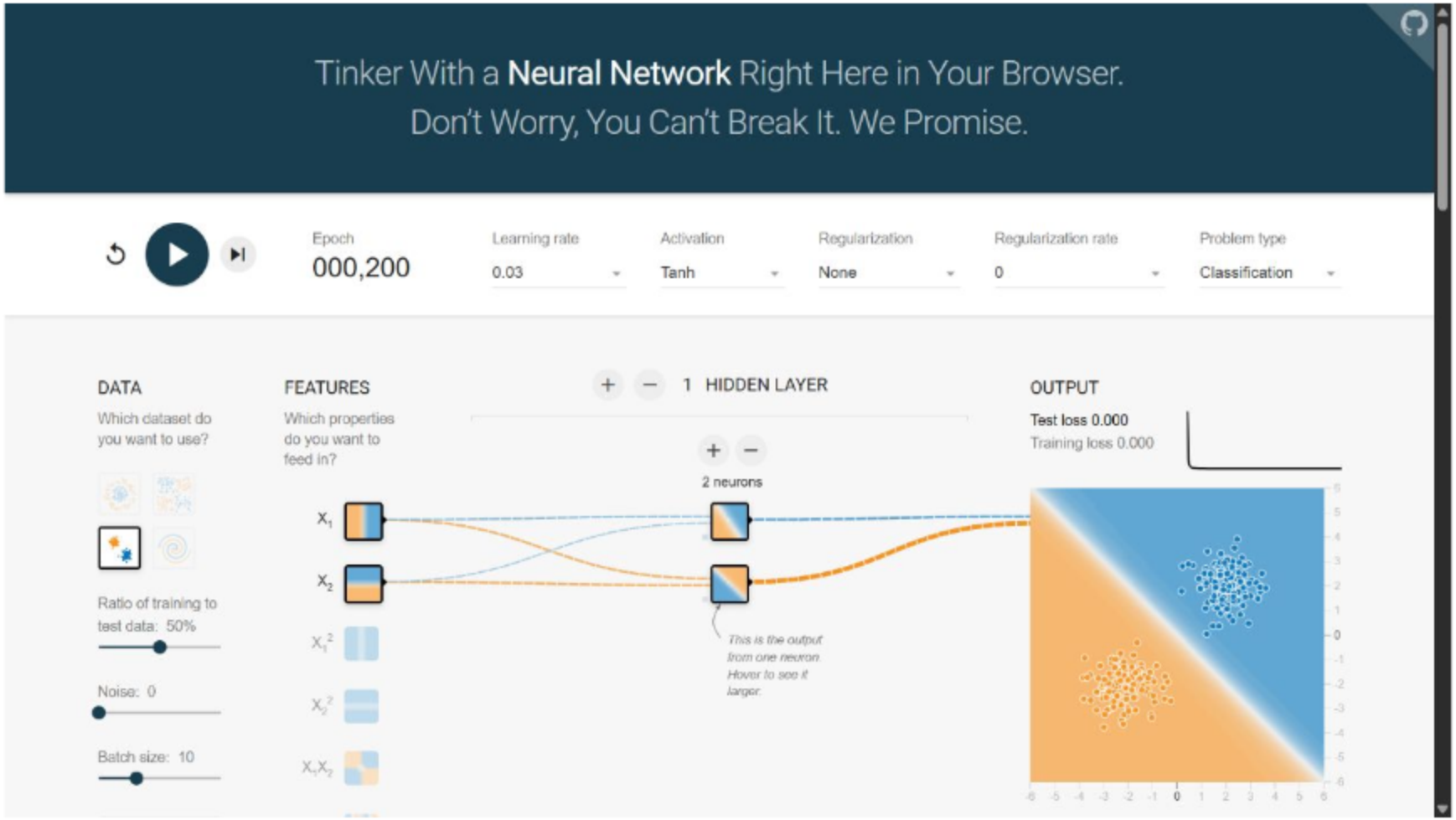


Noise 30
4 then 2 neurons
Train 0.118
Test 0.159

Clean circle
2 hidden neurons
Train 0.218
Test 0.267

Run C shown. More hidden units cannot remove uncertainty caused by overlapping observations.

Dataset difficulty affects the fit



2 hidden neurons
9 parameters

Train 0.000
Test 0.000

Displayed values round to 3 decimals

Run F shown. The same small width struggled with the circle task in run E.

Interpreting the experiments

Same epoch budget, different fits

Each run completed 200 epochs. The training budget alone did not determine performance.

Parameter counts range from 3 to 25. Feature choice changes what those parameters can express.

Browser interaction time includes display overhead. These runs compare epochs and complexity rather than measured compute speed.

Repeated seeds and controlled initialization would strengthen a future comparison.

References

TensorFlow Playground
https://playground.tensorflow.org/

Google ML Crash Course Neural network nodes and hidden layers
https://developers.google.com/machine-learning/crash-course/neural-networks/nodes-hidden-layers

Google ML Crash Course Training using backpropagation
https://developers.google.com/machine-learning/crash-course/neural-networks/backpropagation

TensorFlow Playground source Training loop and output activation
https://github.com/tensorflow/playground/blob/master/src/playground.ts

TensorFlow Playground source Neurons loss and weight updates
https://github.com/tensorflow/playground/blob/master/src/nn.ts

Keras SGD optimizer
https://keras.io/api/optimizers/sgd/

Keras Adam optimizer
https://keras.io/api/optimizers/adam/

Summary

A diagram connects the components

Layers organize neurons. Weights and biases shape each response. Activations enable nonlinear hidden transformations.

Loss measures the mismatch. Backpropagation computes gradients, and optimization updates the parameters.

The observed fits depend on the task, the input representation, and the network capacity.

Visual inspection and held-out loss together make model behavior easier to explain.